

Course: Introduction to Software Engineering Course Code: SW 203

Instructor: Dr. Mohamed Sami Rakha

Team Members: Mostafa Mohamed Helmy (202401133) ,Mohamed Ahmed Ezzat (202401317)

Date: November 6, 2025

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms, and Abbreviations
 - 1.4 References
 - 1.5 Overview
2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints
 - 2.6 User Documentation
 - 2.7 Assumptions and Dependencies
3. Specific Requirements
 - 3.1 Functional Requirements
 - 3.2 Use Case Model
 - 3.2.1 Use Case Diagrams
 - 3.2.2 Use Case Descriptions
 - 3.3 Domain Model
 - 3.3.1 Conceptual Class Diagram
 - 3.3.2 Class Descriptions
 - 3.4 Non-Functional Requirements
 - 3.5 External Interface Requirements
 - 3.5.1 User Interface
 - 3.5.2 Hardware Interface
 - 3.5.3 Software Interface
 - 3.5.4 Communication Interface
4. Appendices
 - 4.1 Data Dictionary
 - 4.2 Glossary
 - 4.3 Non-Functional Requirements Testing

1. Introduction

The full functional and non-functional requirements for ZC Reclaim: The Digital Lost & Found Hub, a web-based application created for Zewail City University, are described in this Software Requirements Specification (SRS) document. As a fundamental artifact for Phase 2 of the software engineering project, this document offers a thorough and lucid understanding of the system to direct stakeholder communication, testing, and development.

1.1 Purpose

This SRS's main goal is to precisely outline ZC Reclaim's requirements so that all parties involved—developers, testers, project managers, and end users—are aware of the system's strengths and weaknesses. This document will be used as a guide during the software development process, ensuring precise implementation, thorough testing, and the successful deployment of a system that satisfies Zewail City's unique requirements.

1.2 Scope

A web-based tool called ZC Reclaim was created to make it easier for Zewail City University to report, search for, claim, and retrieve lost and found items. The system seeks to improve communication, replace manual processes, and increase the effectiveness of item recovery for faculty, staff, and students. Functionalities for the following user roles are included in the scope: Admin, Lost & Found Staff, Registered User, and Guest. User accounts, item recovery processes, and item posts will all be managed by the system. The requirements are the only thing covered in this SRS; implementation specifics and UI design are not covered.

1.3 Definitions, Acronyms, and Abbreviations

Admin (Administrator): An elevated-privilege user role in charge of managing users and systems. A registered user claiming ownership of a discovered item is known as a claim. CRUD stands for Create, Read, Update, Delete; these are essential data management operations. Database: An organized set of information that is electronically stored and accessible. A functional requirement is one that outlines a task that a system or component must be capable of completing. Guest: An unregistered user with restricted access to the ZC Reclaim platform. HTTPS is a secure variant of HTTP that is used for network communication. IEEE 830: A software development standard

Requirements Specification: a document outlining a system's entire behavior that needs to be developed. The interface between a computer system and its user. The web application "The Digital Lost & Found Hub" for Zewail City University is called ZC Reclaim.

1.4 References

[1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications. [2] OMG Unified Modeling Language (UML) Specification, Version 2.5.1. [3] OWASP, 'OWASP Top Ten Project'. [4] W3C, 'Web Content Accessibility Guidelines (WCAG) 2.1'.

1.5 Overview

This SRS document is organized into four main sections, adhering to the IEEE 830-1998 standard.

- **Section 1 (Introduction):** Provides the purpose, scope, definitions, references, and an overview of this document.
- **Section 2 (Overall Description):** Presents a high-level overview of the product, including its perspective, functions, user classes, operating environment, and constraints.
- **Section 3 (Specific Requirements):** Details all functional requirements, use case models, and non-functional requirements for the system.
- **Section 4 (Appendices):** Contains supplementary information, including the data dictionary, glossary, and testing plans for non-functional requirements.

1.5.1 Document Conventions

The IEEE Standard 830-1998 for Software Requirements Specifications is followed in this document. Important customs consist of: Conditions Numbering: "FR-" and a sequential number (FR-1) are used to identify functional requirements. "NFR-" and a sequential number are used to identify non-functional requirements (NFR-1). Requirement Phrasing: "The system shall" is used to state each requirement. Terminology: To maintain uniformity and clarity, certain terms are defined in the Glossary (Appendix D). Tone: The document avoids contractions and colloquial language in favor of a formal, academic tone. The third person is used to write the document.

1.5.2 Intended Audience

This SRS's target audience consists of: Development Team: To comprehend the design and implementation requirements for the system. Testing Team: To create thorough test cases and plans in accordance with specified requirements. Project management: To keep an eye on the status of the project, control its scope, and make sure it meets stakeholder expectations. City of Zewail University Stakeholders: To examine and confirm that the system satisfies their organizational requirements. Future Maintainers: To comprehend the original architecture of the system and the specifications needed for any future improvements.

2. Overall Description

An overview of ZC Reclaim is given in this section, along with details about its background at Zewail City University, its main purposes, its user base, and its operating environment.

2.1 Product Perspective

Zewail City University's current network infrastructure will be integrated with ZC Reclaim, a stand-alone web application. Although it is not a part of a bigger system, it will communicate with university systems for notifications and, if relevant, user authentication. In place of dispersed manual procedures, the system will offer a centralized digital hub for managing lost and found items.

2.2 Product Functions

The core functions of ZC Reclaim include:

- **Item Reporting:** Allowing registered users to report lost or found items with detailed descriptions and images.
- **Item Searching and Browsing:** Enabling all users (guests and registered) to search and browse existing lost and found item posts.
- **Item Claiming:** Providing a mechanism for registered users to claim reported lost items, including verification processes.
- **Item Management:** Allowing Admin and Lost & Found Staff to validate posts, manage reported items, and update their status (e.g., returned).
- **User Management:** Enabling administrators to manage user accounts and roles.
- **Notification System:** Facilitating communication regarding item matches, claims, and recovery status.

2.3 User Classes and Characteristics

ZC Reclaim will support four distinct user classes, each with specific characteristics and privileges:

- **Guest:**
 - **Characteristics:** Unregistered users of the platform.
 - **Privileges:** The system shall permit guests to browse lost and found items. The system shall permit guests to search for lost and found items.

- **Registered User:**
 - **Characteristics:** Students, staff, and faculty of Zewail City University who have registered and logged into the system.
 - **Privileges:** The system shall allow registered users to report lost items. The system shall allow registered users to report found items. The system shall allow registered users to claim lost items. The system shall allow registered users to track the recovery status of their claimed or reported items.
- **Admin:**
 - **Characteristics:** Designated university administrators with elevated privileges.
 - **Privileges:** The system shall enable administrators to manage user accounts. The system shall enable administrators to delete spam posts. The system shall enable administrators to approve or reject item claims.
- **Lost & Found Staff:**
 - **Characteristics:** Official personnel responsible for the physical Lost & Found department at Zewail City University.
 - **Privileges:** The system shall allow Lost & Found staff to confirm returned items. The system shall allow Lost & Found staff to manage the physical storage status of items within the system.

2.4 Operating Environment

ZC Reclaim will function as a web-based application that can be accessed on a variety of devices (desktops, laptops, tablets, and smartphones) using common web browsers (such as Chrome, Firefox, Edge, and Safari). A reliable and expandable infrastructure will be used to host the system on university servers. To store data, the backend will communicate with a relational database.

2.5 Design and Implementation Constraints

- **Technology Stack:** The specific technology stack (e.g., programming languages, frameworks, database) will be determined during the design phase, but it must be compatible with Zewail City University's existing IT infrastructure and security policies.
- **Security Standards:** The system shall adhere to Zewail City University's security policies and industry best practices for web application security, including data encryption and access control.
- **Scalability:** The system shall be designed to accommodate a growing number of users and item posts without significant performance degradation.
- **Accessibility:** The user interface shall conform to web accessibility guidelines (e.g., WCAG 2.1) to ensure usability for all users.
- **Language:** The system shall be developed primarily in English.

2.6 User Documentation

Comprehensive user documentation will be provided to assist all user classes in utilizing ZC Reclaim effectively. This documentation will include:

- User manuals for Registered Users, Admins, and Lost & Found Staff.
- Frequently Asked Questions (FAQs) section within the application.
- Context-sensitive help features.

2.7 Assumptions and Dependencies

- **Internet Connectivity:** Users are assumed to have reliable internet access to utilize the web application.
- **University Network Infrastructure:** The university's network infrastructure is assumed to be stable and capable of supporting the application's traffic.

- **User Data Accuracy:** The accuracy of user-provided information (e.g., item descriptions, contact details) is assumed.
- **Staff Availability:** Dedicated Lost & Found staff will be available to manage physical items and update their status within the system.
- **Integration with University Systems:** Any required integration with existing university authentication systems will be supported by the university IT department.

3. Specific Requirements

This section details the functional requirements of ZC Reclaim, organized by major system features. Each feature includes an introduction and a list of functional requirements, all beginning with "The system shall...".

3.1 Functional Requirements

User Authentication and Authorization

This feature manages user registration, login, and ensures that users can only access functionalities appropriate to their assigned roles.

User Accounts & Access

- **3.1.1:** New users can easily sign up for an account using their official Zewail City University email address.
- **3.1.2:** We'll send a quick verification email to confirm your account creation, so keep an eye on your inbox!
- **3.1.3:** Once registered, you can log in securely with your verified email and password.
- **3.1.4:** Forgot your password? No worries! We have a simple way for you to reset it.
- **3.1.5:** Our system smartly manages access based on who you are: a Guest, a Registered User, an Admin, or a Lost & Found Staff member.
- **3.1.6:** We make sure that everyone only sees and does what they're authorized to, keeping your information safe and sound.

Item Reporting

This feature enables registered users to create and submit posts for both lost and found items.

User System Requirements

- **3.1.7:** You'll be able to create a new "Lost Item" post if you're a registered user.
- **3.1.8:** When creating a "Lost Item" post, we'll need a few details: the item's name, a good description, roughly when and where it was lost, and how we can reach you.
- **3.1.9:** You can add up to three pictures to your "Lost Item" post.
- **3.1.10:** If you're a registered user, you can also create a new "Found Item" post.
- **3.1.11:** For a "Found Item" post, we'll ask for the item's name, a detailed description, about when and where you found it, and your contact information.
- **3.1.12:** Feel free to upload up to three images for your "Found Item" post.
- **3.1.13:** Every lost or found item post will get its own special ID number.
- **3.1.14:** We'll automatically note the date and time each post is created.

Item Searching and Browsing

This feature provides mechanisms for all users to find lost or found items.

- **3.1.15:** Anyone, whether just visiting or a registered member, should be able to look through all the active posts about lost and found items.
- **3.1.16:** Every user should be able to search for item posts using keywords, like the item's name or a

description.

- **3.1.17:** Users should be able to narrow down their search results by whether the item was lost or found.
- **3.1.18:** Users should also be able to filter their search results by the date the item was posted.
- **3.1.19:** You should be able to filter search results by location, for example, a specific building or area on campus.
- **3.1.20:** The system will show a list of matching item posts with a quick summary, including things like the item's name, the date it was posted, its location, and a small picture.
- **3.1.21:** Users should be able to click on an item post to see all the details, including all the information and pictures provided.

Item Claiming

This feature facilitates the process for registered users to claim a lost item they have identified through the system.

Found Item Claim Process:

- **3.1.22:** As a registered user, you can initiate a claim for any "Found Item" that matches something you've lost.
- **3.1.23:** To verify you're the rightful owner, we'll ask you for some unique details about the item that aren't visible in the public post.
- **3.1.24:** We'll let the person who posted the "Found Item" know that someone has made a claim.
- **3.1.25:** Our Lost & Found Staff will also be informed about the new claim so they can help manage the process.
- **3.1.26:** You'll be able to keep an eye on the progress of your claim every step of the way.
- **3.1.27:** The person who found the item can chat with you (through a secure and moderated channel) to confirm all the details and ensure it's yours.

Item Management (Admin and Lost & Found Staff)

This feature provides tools for administrators and Lost & Found staff to manage item posts and their recovery status.

- **3.1.28:** Admins and Lost & Found Staff should be able to see all the items that have been reported as lost or found.
- **3.1.29:** Admins and Lost & Found Staff should be able to change the details of any item that's been posted.
- **3.1.30:** Admins and Lost & Found Staff should be able to mark an item as "Returned" once its owner has successfully gotten it back.
- **3.1.31:** Items marked "Returned" should automatically be hidden from public view after a certain amount of time, which can be set by the system.
- **3.1.32:** Admins and Lost & Found Staff should be able to delete item posts (for example, if they're spam or duplicates).
- **3.1.33:** Admins and Lost & Found Staff should be able to manage the status of claims (like approving or rejecting them).
- **3.1.34:** The system should let the people involved (the person claiming the item and the person who posted it) know what happened with a claim.

User Management (Admin)

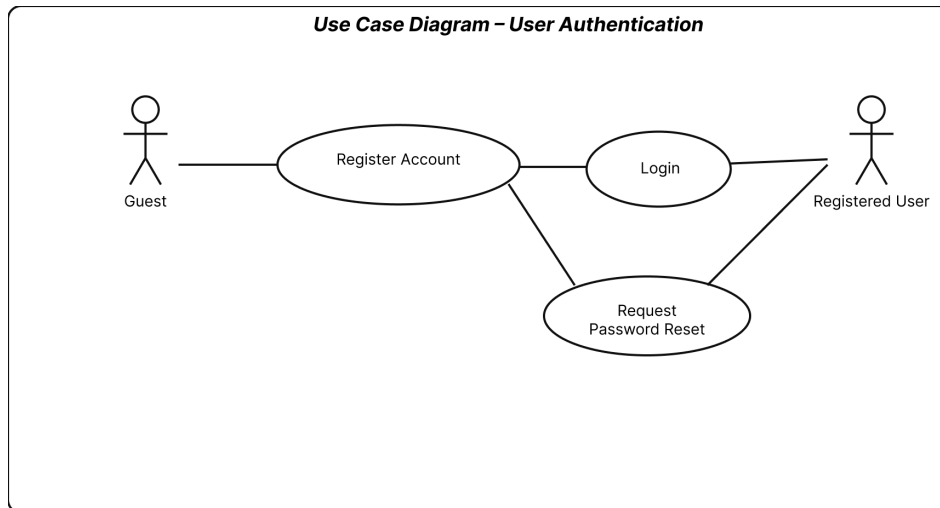
This feature enables administrators to manage user accounts and their associated roles.

- **3.1.35:** As an administrator, you'll be able to see a full list of everyone registered on the system.
- **3.1.36:** Administrators will have the ability to easily search for any specific user they need to find.
- **3.1.37:** Administrators can change what users are allowed to do by modifying their roles (for example, giving someone the 'Lost & Found Staff' role).
- **3.1.38:** Administrators will be able to turn user accounts on or off as needed.
- **3.1.39:** If a user forgets their password, administrators will be able to reset it for them.

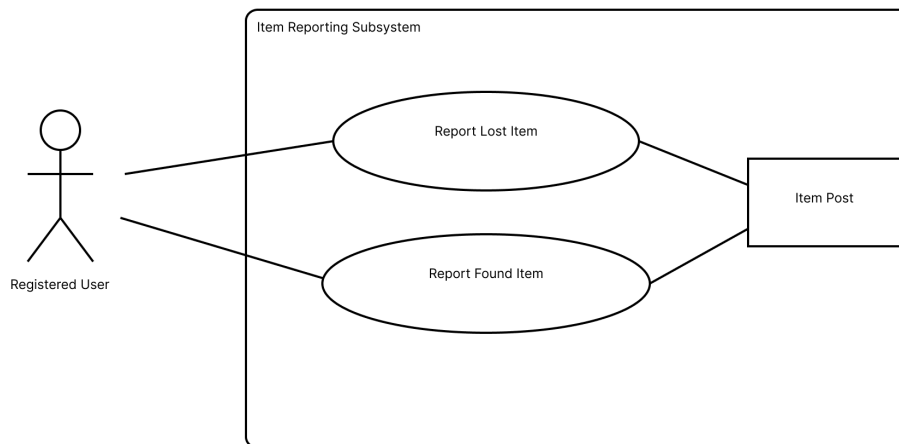
3.2 Use Case Model

3.2.1 Use Case Diagrams

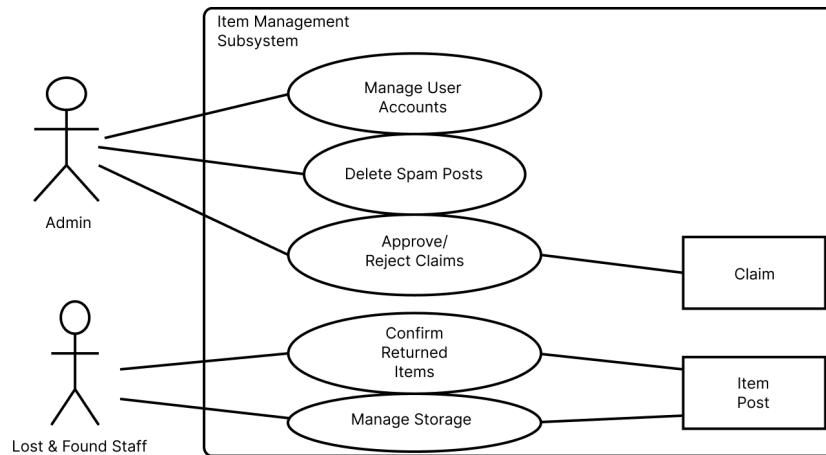
Use Case Diagram: User Authentication



Use Case Diagram: Item Reporting



Use Case Diagram: Item Management



3.2.2 Use Case Descriptions

1. Use Case Diagram: User Authentication

Use Case: Register Account

- **ID:** UC-02
- **Actor:** Guest
- **Description:** Allows a Guest to create a verified account using a ZC email.
- **Main Flow:**
 - Guest submits the registration form with a ZC email.
 - System validates data and sends a verification email.
 - User clicks the link in the email.
 - System validates the token and updates the user's status to "verified."
- **Alternative Flows:**
 - **Invalid Data:** If data is invalid (e.g., email exists), system displays an error.
 - **Invalid Token:** If the token is invalid/expired, system displays an error.
- **Postcondition:** A new, verified "RegisteredUser" account is created.

Use Case: Login

- **ID:** UC-03
- **Actors:** Guest, Registered User
- **Description:** Allows an existing user to gain authenticated access.
- **Main Flow:**
 - User submits the login form with their email and password.
 - System validates credentials.
 - If successful, system creates a secure session and grants access based on user role.
- **Alternative Flows:**
 - **Invalid Credentials:** System displays a generic error message.
 - **Account Inactive:** If account is unverified or deactivated, system displays an error.
 - **Forgot Password:** User initiates the "Request Password Reset" use case.
- **Postcondition:** The user is authenticated and has a secure session.

Use Case: Request Password Reset

- **ID:** UC-07
- **Actors:** Guest, Registered User
- **Description:** Allows a user who has forgotten their password to securely reset it.
- **Main Flow:**
 - User submits their email via the "Forgot Password" form.
 - System verifies the email exists and sends a password reset link.
 - User clicks the link, validates the token, and submits a new password.
 - System validates and updates the new password hash in the database.
- **Alternative Flows:**
 - **Email Not Found:** System displays a generic confirmation (for security).
 - **Invalid Token:** System displays an error and prompts the user to restart.
- **Postcondition:** The user's password is changed.

2. Use Case Diagram: Item Reporting

Use Case: Report Lost Item

- **ID:** UC-01
- **Actor:** Registered User
- **Description:** Allows a Registered User to create a post for a lost item.
- **Main Flow:**
 - Registered User selects "Report Lost Item".
 - User submits the form with item details (name, description, location) and optional images.
 - System validates the data and creates a new "Lost Item" post with an "Active" status.
- **Alternative Flows:**
 - **Invalid Data:** If required fields are missing, system displays an error.
- **Postcondition:** A new "Lost Item" post is created and visible to others.

Use Case: Report Found Item

- **ID:** UC-04
- **Actor:** Registered User
- **Description:** Allows a Registered User to create a post for a found item.
- **Main Flow:**
 - Registered User selects "Report Found Item".
 - User submits the form with item details (name, description, location) and optional images.
 - System validates the data and creates a new "Found Item" post with an "Active" status.
- **Alternative Flows:**
 - **Invalid Data:** If required fields are missing, system displays an error.
- **Postcondition:** A new "Found Item" post is created and visible to others.

3. Use Case Diagram: Item Management

Use Case: Manage User Accounts

- **ID:** UC-06
- **Actor:** Admin
- **Description:** Allows an Admin to manage user roles and status.
- **Main Flow:**
 - Admin navigates to User Management and selects a user.
 - Admin chooses an action: modify role , deactivate/activate account , or reset password.
 - System updates the user's record and confirms the change.

- **Alternative Flows:**
 - **Self-Modification:** System prevents an Admin from deactivating their own account.
- **Postcondition:** The selected user's account details are updated.

Use Case: Delete Spam Posts

- **ID:** UC-09
- **Actor:** Admin
- **Description:** Allows an Admin to permanently delete inappropriate or spam posts.
- **Main Flow:**
 - Admin identifies a spam post and selects "Delete".
 - Admin confirms the deletion.
 - System permanently removes the item post and logs the action.
- **Alternative Flows:**
 - **Canceled:** If Admin cancels, the post is not deleted.
- **Postcondition:** The target item post is permanently removed.

Use Case: Approve/Reject Claims

- **ID:** UC-08
- **Actor:** Admin
- **Description:** Allows an Admin to review and resolve a pending item claim.
- **Main Flow:**
 1. Admin selects a "Pending" claim to review.
 2. Admin reviews the claim details and selects "Approve" or "Reject".
 3. System updates the claim status and notifies the relevant users.
- **Postcondition:** The claim's status is updated to "Approved" or "Rejected."

Use Case: Confirm Returned Items

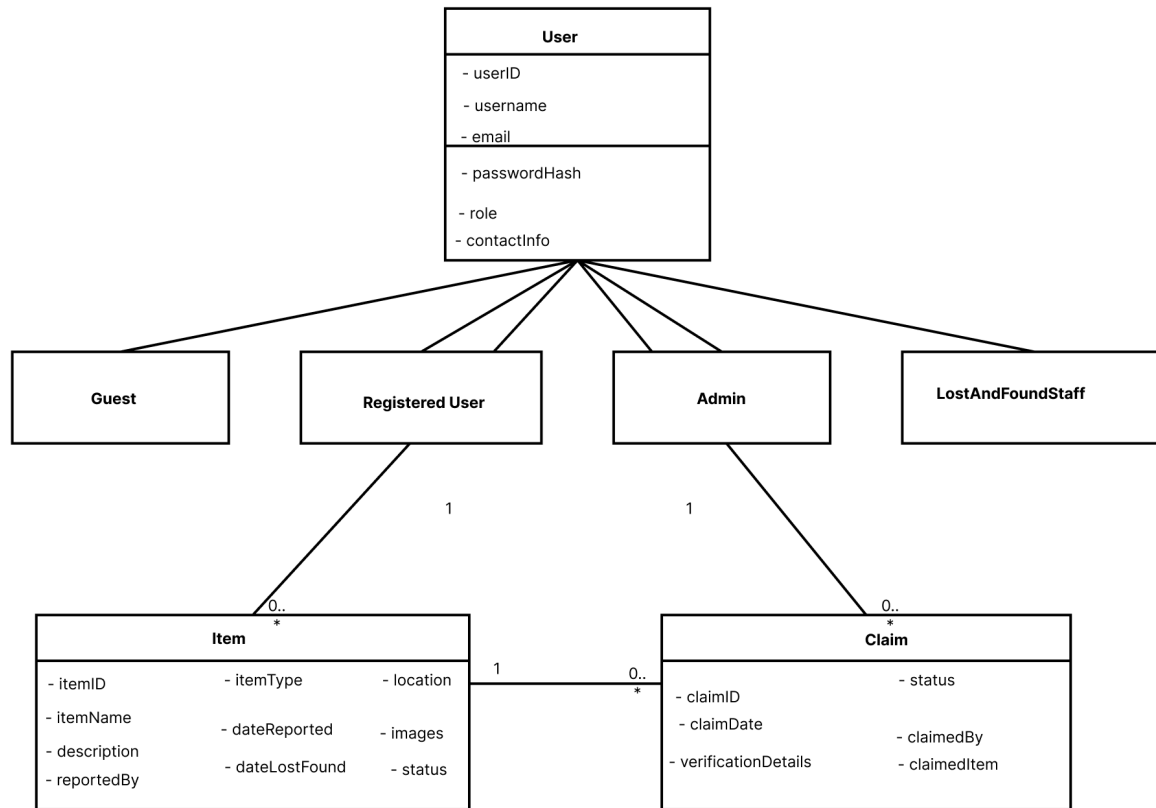
- **ID:** UC-10
- **Actor:** Lost & Found Staff
- **Description:** Allows staff to mark an item as "Returned" after physical handoff.
- **Main Flow:**
 - Lost & Found Staff locates the item post.
 - Staff selects "Mark as Returned" and confirms the action.
 - System updates the item's status to "Returned," and the post is archived.
- **Alternative Flows:**
 - **Canceled:** If Staff cancels, the item's status is unchanged.
- **Postcondition:** The item's status is "Returned" and it is hidden from public view.

Use Case: Manage Storage

- **ID:** UC-11
- **Actor:** Lost & Found Staff
- **Description:** Allows staff to update an item's physical storage location for internal tracking.
- **Main Flow:**
 1. Lost & Found Staff selects an item post and chooses "Manage Storage".
 2. Staff enters the item's physical location (e.g., "Bin-A14") in a private field.
 3. System saves this internal-tracking information.
- **Postcondition:** The item's physical storage information is updated.

3.3 Domain Model

3.3.1 Conceptual Class Diagram



3.3.2 Class Descriptions.

- **User**: This is the base class for all entities that interact with the system. It stores common attributes such as userID, username, email, passwordHash, and role. It serves as a parent class for all specific user types.
- **Guest**: A specialization of User that represents an unauthenticated (unregistered) visitor. Guests have limited privileges, such as the ability to browse and search for items.
- **Registered User**: A specialization of User representing an authenticated member of Zewail City University (student, staff, or faculty). This user can perform all guest functions, as well as report lost items, report found items, and claim found items.
- **Admin**: A specialization of User with elevated system privileges. The Admin is responsible for system oversight, including managing user accounts and moderating content.
- **LostAndFoundStaff**: A specialization of User representing official university personnel. This role manages the physical workflow of items, such as confirming when an item is returned and managing the physical storage status of items.
- **Item**: Represents a single item post within the system. It contains all details about a lost or found object, including its itemName, description, itemType (Lost/Found), location, images, and current status. Each Item is reported by one Registered User.

- **Claim:** Represents a formal assertion of ownership made by a Registered User for a "Found Item" post. This class stores the claimID, verificationDetails provided by the claimant, and the status of the claim. Each Claim is associated with one Item and one Registered User (the claimant).

3.4 Non-Functional Requirements

This section describes the non-functional requirements that define the quality attributes of ZC Reclaim.

Performance

- **3.4.1:** We want our main pages, like the homepage, search results, and item details, to load in under 3 seconds, even when up to 100 people are using the system at once.
- **3.4.2:** When someone posts an item, we want it to be processed and submitted within 2 seconds.
- **3.4.3:** We aim for search queries to return results within 2 seconds, even with a database of up to 10,000 item posts.
- **3.4.4:** Our system should be able to handle at least 500 people using it at the same time without slowing down noticeably.

Security

- **3.4.5:** We will protect all your personal information and item details from anyone who shouldn't see, change, or share them.
- **3.4.6:** We will use a secure connection (HTTPS) for all data you send and receive, so it's always encrypted.
- **3.4.7:** We will ask you to create strong passwords with a good length, different types of characters, and we'll remind you to change them regularly.
- **3.4.8:** Your passwords will be stored securely, scrambled and mixed with unique values to keep them safe.
- **3.4.9:** We will make sure that people only have access to what they need based on their role in the system.
- **3.4.10:** We will protect the system from common online attacks like SQL injection, XSS, and CSRF.
- **3.4.11:** We will keep a record of all important security events, like login attempts and data changes, so we can always review them.
- **3.4.12:** If you're inactive for a certain amount of time (for example, 30 minutes), we'll automatically log you out to keep your account secure.

Reliability

- **3.4.13:** We'll make sure the system is up and running 99.9% of the time, except for when we have to do planned maintenance.
- **3.4.14:** The system will have smart ways to handle errors and keep detailed logs, so we can quickly find and fix any problems.
- **3.4.15:** We'll protect your data, ensuring that no information about items or users gets corrupted or lost.
- **3.4.16:** In case of a major disaster, we have a plan to get everything back online and restore all data.
- **3.4.17:** We'll automatically back up the database every single day.

Maintainability

3.4.18: We'll make sure our code is easy to understand, broken into logical parts, and follows our team's best practices.

3.4.19: We'll design the system so that different parts can be changed without messing up everything else.

3.4.20: We'll include thorough logging to help us find and fix issues, and to keep an eye on how things are running.

3.4.21: We'll make it straightforward to set up and run the system in various environments, like for testing or live use.

Usability

3.4.22: We want our system to be so easy to use that new users can jump right in with very little training.

3.4.23: When something goes wrong, the system should tell you exactly what happened and how to fix it, clearly and simply.

3.4.24: No matter where you are in the system, you should always feel at home. Navigation and layout will be consistent across all pages.

3.4.25: We're going to make common tasks, like reporting an item or searching, as quick and painless as possible, with the fewest steps.

3.4.26: You'll always know what's happening when you interact with the system, thanks to clear visual cues and feedback.

Portability

3.4.27 Operating System Compatibility:

Our system is designed to work smoothly on various operating systems, including popular choices like Linux and Windows Server. As long as the operating system supports the technologies we use, you're good to go!

3.4.28 Database Compatibility:

We've made sure our system's database plays nicely with standard relational database systems, so you shouldn't have any trouble connecting.

3.4.29 Browser Compatibility:

The front end of our system is built to look and feel great on all modern web browsers. No need for extra plugins or extensions – it just works!

Other Requirements

3.4.30: We'll make sure our system follows all of Zewail City University's rules about keeping your data private and managing information carefully.

3.4.31: We'll keep a detailed record of important actions taken by our Admin and Lost & Found Staff, so we can always see what happened.

3.5 External Interface Requirements

This section describes how ZC Reclaim will interact with users, hardware, other software systems, and communication protocols.

3.5.1 User Interface

The user interface for ZC Reclaim will be a web-based graphical user interface (GUI).

- **User-Friendly Design:** We're building a user interface that feels natural and easy to use, with a consistent look and feel across the entire system. Our goal is to make your experience smooth and enjoyable.
- **Easy Navigation:** You'll always know where you are and how to get where you want to go, thanks to clear menus and helpful "breadcrumbs" that show your path.

- **Works Everywhere:** Whether you're on your desktop, tablet, or phone, the interface will adapt seamlessly to fit your screen.
- **Smart Input:** When you need to type something, you'll see clear labels, helpful suggestions, and instant feedback if something's not quite right.
- **Clear Feedback:** You'll always know what's happening – whether your submission was successful, if there's an error, or if something is loading – with clear visual cues.
- **Accessible to All:** We're committed to making sure everyone can use our system, including those with disabilities. We're following accessibility guidelines to ensure a truly inclusive experience.

3.5.2 Hardware Interface

ZC Reclaim will primarily interact with standard client-side hardware through web browsers.

- **Client Devices:** The system shall be compatible with personal computers, laptops, tablets, and smartphones.
- **Input Devices:** The system shall support standard input devices such as keyboards, mice, and touchscreens.
- **Display Devices:** The system shall be viewable on various display resolutions and sizes.
- **Image Upload:** The system shall interface with client device cameras or file systems for image uploads.

3.5.3 Software Interface

ZC Reclaim may interface with external software systems.

- **Email Service:** We'll connect with an email service to send important messages like account verification, password reset links, and claim notifications.
- **Database Management System:** Our system will work with a relational database (MySQL) to store and retrieve all the necessary data.
- **Image Storage Service:** We might also link up with a cloud-based or local storage service to keep all the uploaded item images safe and accessible.

3.5.4 Communication Interface

The communication between the client (web browser) and the server will be over standard internet protocols.

- **HTTP/HTTPS:** All communication between the client and the server shall utilize HTTPS for secure data transmission.
- **Web Sockets (Optional):** For real-time notifications or chat functionalities between users and staff, the system may utilize Web Sockets.
- **APIs:** Any external integrations will be conducted via well-defined Application Programming Interfaces (APIs).

4. Appendices

4.1 Data Dictionary

User Table

Data Element	Description	Data Type	Constraints/Format
userID	Unique identifier for each user.	UUID	"Primary Key, Auto-generated"
username	User's chosen username.	String	"Unique, Max 50 characters"
email	User's Zewail City University email address.	String	"Unique, Valid email format (@zewailcity.edu.eg)"
passwordHash	Hashed and salted password for user authentication.	String	Min 60 characters (for strong hash)
role	User's assigned role in the system.	Enum	"{Guest, RegisteredUser, Admin, LostAndFoundStaff}"
contactInfo	"User's preferred contact information (e.g., phone number)."	String	"Optional, Max 100 characters"

Item Table

Data Element	Description	Data Type	Constraints/Format
itemID	Unique identifier for each item post.	UUID	"Primary Key, Auto-generated"
itemName	Name of the lost or found item.	String	"Required, Max 100 characters"
description	Detailed description of the item.	Text	"Required, Max 1000 characters"

itemType	Indicates whether the item is lost or found.	Enum	"{Lost, Found}"
dateReported	Date and time when the item was reported.	DateTime	"Auto-generated, Cannot be in the future"
dateLostFound	Approximate date and time when the item was lost or found.	DateTime	"Required, Cannot be in the future"
location	Specific location where the item was lost or found on campus.	String	"Required, Max 200 characters"
images	List of URLs to uploaded images of the item.	List<URL>	Max 3 URLs
status	Current status of the item post.	Enum	"{Active, Claimed, Returned, Archived}"
reportedBy	Foreign key referencing the userID of the user who reported the item.	UUID	Foreign Key to User.userID

Claim Table

Data Element	Description	Data Type	Constraints/Format
claimID	Unique identifier for each claim.	UUID	"Primary Key, Auto-generated"
claimDate	Date and time when the claim was made.	DateTime	

verificationDetails	Additional details provided by the claimant for verification.	Text	"Required, Max 500 characters"
claimedBy	Foreign key referencing the userID of the user who made the claim.	UUID	Foreign Key to User.userID
claimedItem	Foreign key referencing the itemID of the item being claimed.	UUID	Foreign Key to Item.itemID

4.2 Glossary

(All definitions, acronyms, and abbreviations are located in section 1.3)

4.3 Non-Functional Requirements Testing

ID	Requirement	Test Method	Success Criteria
NFR-1 Performance	Homepage loads $\leq 3s$ under normal load (≤ 100 concurrent users).	Load test with 100 VUs for 10 minutes; measure p95 response time.	p95 ≤ 3000 ms; error rate $< 1\%$.
NFR-2 Search Performance	"Search returns $\leq 2s$ for 10,000 posts."	Seed DB with 10k items; run 1000 mixed search queries; measure p95.	p95 ≤ 2000 ms; CPU $< 80\%$ avg.
NFR-5 Security	Protect against OWASP Top 10 risks.	Static code scan + dynamic scan + manual pen-test.	All Critical/High findings remediated; no unauth data access.
NFR-8 Password Storage	Passwords stored salted+hashed.	Inspect configuration and DB values; attempt rainbow attack.	"Only salted+hashed values (e.g., bcrypt/argon2); no plaintext."

NFR-11 Audit Logging	Security events are logged.	"Trigger login attempts, password reset, role changes; inspect logs."	"All events appear with timestamp, actor ID, IP."
NFR-13 Availability	99.9% monthly uptime excluding maintenance.	Monitor uptime via external probe 1-min interval.	Downtime $\leq$ 43 minutes/month.
NFR-22 Usability	UI is intuitive and accessible.	Heuristic evaluation + 5-user test; WCAG 2.1 AA quick checks.	$\geq$ 80% tasks completed without help; $\leq$ 2 critical UX issues.
NFR-27 Portability	Runs on Linux and Windows Server.	Deploy on both OS in staging; run smoke tests.	All smoke tests pass on both OS.